



NU MOA



React Styling and Properties

FOUNDED FOR FILIPINOS, BUILT FOR THE NATION

At the end of the lesson(s), students must be able to:

- Understand what props are and how they work in React.
- Learn how to pass data to React components using props.
- Understand how to use styling in React.
- Learn how to style React components using CSS.

What is React Props?



NU MOA



Props means **Properties**.

Props are used to **send information from one component to another**.

Think of Props as giving information to a component.

Example:

Instead of creating many Student components, we create **one Student component** and simply change its information using Props.

Why Do We Use Props?



NU MOA



Props help us:

- Reuse components
- Display different information
- Make code cleaner
- Avoid writing the same code again

Example of Without using Props



NU MOA



Every student has its own separate component.

```
<Student1 />
<Student2 />
<Student3 />
```

Inside each component:

```
// Student1.jsx
function Student1() {
  return <h2>Juan Dela Cruz - BSIT</h2>;
}
```

```
// Student2.jsx
function Student2() {
  return <h2>Maria Santos - BSCS</h2>;
}
```

```
// Student3.jsx
function Student3() {
  return <h2>Pedro Reyes - BSIS</h2>;
}
```

Problem

- Too much repeated code
- Difficult to maintain
- Not reusable

Example of With using Props



NU MOA



Create only one Student component and pass different data.

```
<Student name="Juan Dela Cruz" course="BSIT" />
<Student name="Maria Santos" course="BSCS" />
<Student name="Pedro Reyes" course="BSIS" />
```

Advantages

- One reusable component
- Less code
- Easier to update
- Different data can be displayed using Props

Inside the Student component:

```
function Student(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>{props.course}</p>
    </div>
  );
}
```

Result:

```
Juan Dela Cruz
BSIT

Maria Santos
BSCS

Pedro Reyes
BSIS
```

Props can pass almost any JavaScript value.

Data Types	Example
String	name="Christian"
Number	age={22}
Boolean	isOnline={true}
Array	subjects=[{"React","Node","PHP"}]
Object	student={{ name:"Christian", course:"BSIT" }}
Function	handleClick={saveStudent}

Passing Multiple Props



NU MOA



Example:

```
<Student  
  name="Christian"  
  course="BSIT"  
  year={3}  
  section="IT-3A"  
  gender="Male"
```

Inside the components:

```
props.name  
  
props.course  
  
props.year  
  
props.section  
  
props.gender
```

Destructuring Props



NU MOA



Instead of writing props

```
props.name  
props.course  
props.year
```

Advantages

- Cleaner
- Easier to read
- Less typing

Do this:

```
function Student({name,course,year}){
```

And now:

```
<h2>{name}</h2>  
  
<p>{course}</p>  
  
<p>{year}</p>
```

What is the children Prop?

In React, every component can receive a special built-in prop called children.

The children prop represents any JSX elements or content placed between the opening and closing tags of a component.

Instead of passing data as an attribute (such as `name="Christian"`), you place the content directly inside the component tags.

This allows components to act as containers or wrappers for other components or elements.

Example of Children Prop



NU MOA



App.jsx (Parent Component)

```
import Card from "../Card";

function App() {
  return (
    <Card>
      Hello Students!
    </Card>
  );
}
```

Card.jsx (Child Component)

```
function Card({ children }) {
  return (
    <div>
      {children}
    </div>
  );
}

export default Card;
```

Output

Hello Students!

What is React Prop Validation?

- React components receive data through **props**.
- Sometimes, developers accidentally pass the wrong type of data.
- **Prop Validation** is the process of checking whether the data passed to a component has the correct type.
- React performs this validation using the **PropTypes** library.

Why Do We Need Prop Validation?

- Imagine we have a Student component.
- It expects the student's **age** to be a number.

Correct:

```
<Student age={20} />
```

Wrong:

```
<Student age="Twenty" />
```

- The second example passes a **string** instead of a **number**.
- Without validation, this mistake may cause unexpected behavior.

Benefits of Prop Validation

- ✓ Detects incorrect data types
- ✓ Makes debugging easier
- ✓ Improves code quality
- ✓ Makes components easier to reuse
- ✓ Helps other developers understand what data a component expects

Installing PropTypes

First, install the package.

```
npm install prop-types
```

Then import it into your component.

```
import PropTypes from "prop-types";
```

Example PropTypes



NU MOA



Student.jsx

```
import PropTypes from "prop-types";

function Student({ name, age }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
    </div>
  );
}

Student.propTypes = {
  name: PropTypes.string,
  age: PropTypes.number
};

export default Student;
```

App.jsx

```
<Student
  name="Christian"
  age={22}
/>
```

Output:

```
<Student
  name="Christian"
  age={22}
/>
```

Common PropTypes



PropType	Description	Example
string	Text	"Christian"
number	Numbers	25
bool	True or False	true
array	List of values	["Math","React"]
object	JavaScript Object	{name:"John"}
func	Function	handleClick
node	JSX Content	Hello
element	React Element	<Button />

Sometimes a prop must always be provided.

Using **isRequired**.

```
Student.propTypes = {  
  name: PropTypes.string.isRequired,  
  age: PropTypes.number.isRequired  
}
```

Default Props



NU MOA



Default Props provide a default value when no prop is passed.

```
Student.defaultProps = {  
  course: "BSIT"  
}
```

If the parent writes

```
<Student />
```

Output

```
Course: BSIT
```

What is React CSS Styling?

CSS (Cascading Style Sheets) is used to make a website look attractive. Without CSS, a React application only displays plain HTML elements. React allows us to style components in different ways.

Example (Without CSS)

```
function App() {  
  return (  
    <h1>Hello React</h1>  
  );  
}
```

Output

Hello React

Plain text with the browser's default style.

What is React CSS Styling?

CSS (Cascading Style Sheets) is used to make a website look attractive. Without CSS, a React application only displays plain HTML elements. React allows us to style components in different ways.

Example (With CSS)

```
function App() {  
  return (  
    <h1 className="title">  
      Hello React  
    </h1>  
  );  
}
```

CSS

```
.title{  
  color: blue;  
  font-size:40px;  
}
```

Output:

Hello React

React supports several styling methods.

- External CSS
- Inline CSS
- CSS Modules
- Tailwind CSS

Ways to Style React Components



NU MOA



Method 1: External CSS

- This is the most common styling method.
- The CSS is stored in a separate .css file.

Example:

App.css

```
.title{
  color: blue;
  font-size:40px;
  text-align:center;
}
```

App.jsx

```
import "../App.css";

function App(){
  return(
    <h1 className="title">
      Hello React
    </h1>
  );
}

export default App;
```

Ways to Style React Components



NU MOA



Method 2: Inline CSS

- Instead of creating a CSS file,
- the styles are written directly inside the JSX.

Important Rules in Inline CSS

- React uses **camelCase**.
- React treats the style object as a JavaScript object.
- JavaScript object properties cannot contain hyphens.

Example:

CSS Property:	React Property:
background-color	backgroundColor

```
function App(){  
  return(  
    <h1 style={{  
      color:"red",  
      fontSize:"40px",  
      textAlign:"center"  
    }}>  
      Hello React  
    </h1>  
  )  
}
```

Ways to Style React Components

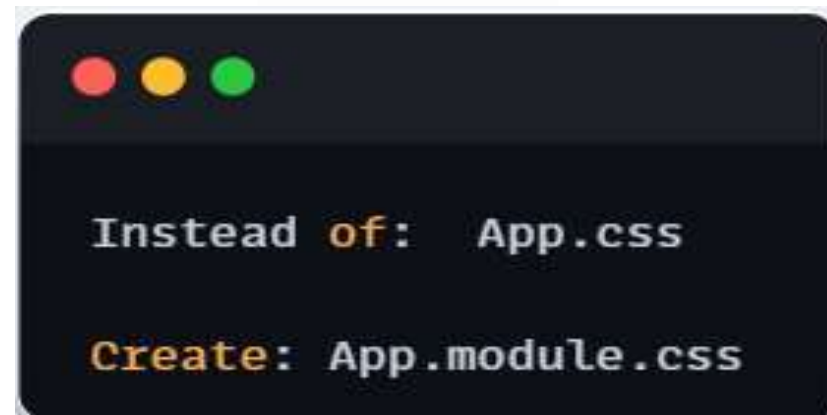


NU MOA



Method 3: CSS Modules

CSS Modules prevent style conflicts.



Why CSS Modules?

Imagine two components **Student.css** and **Teacher.css** both contain **.title** without CSS modules they may conflict. But, with CSS modules, React automatically creates unique class names.

CSS Module Example

```
App.module.css
.title{
  color:green;
}

App.jsx
import styles from "./App.module.css";

function App(){
  return(
    <h1 className={styles.title}>
      Hello React
    </h1>
  )
}
```

Ways to Style React Components



NU MOA



Method 4: Tailwind CSS

Tailwind is a utility-first CSS framework. Instead of writing CSS, you write utility classes.

Step 1: Install Tailwind CSS

```
npm install tailwindcss @tailwindcss/vite
```

Step 2: Configure Vite → Open vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";
```

```
export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
});
```

Step 3: Import Tailwind in CSS → Step 3: Import Tailwind in CSS

```
@import "tailwindcss";
```

Step 4: Use Tailwind in React → Open App.jsx

```
import "../App.css";

function App() {
  return (
    <h1 className="text-4xl text-blue-600 font-bold">
      Hello React
    </h1>
  );
}

export default App;
```

Step 5: Run the Project

```
npm run dev
```



NU MOA



Thank You !!



FOUNDED FOR FILIPINOS, BUILT FOR THE NATION